

Unit-3

Notes

Subject: Object Oriented Programming

Subject Code: BMC202

Course: MCA

Semester: 2

Topic -1: Define Exceptions and its types

Topic-2: Exception Hierarchy

Topic-3: Throwing and Catching exceptions

Topic-4: Built-in exceptions

Topic-5: Creating own exceptions

Topic-6: Stack Trace Element in java

Topic-7: Input-output basics Byte Streams and Character Stream

Topic-8: Reading and Writing

Topic-9: Console Reading and Writing Files

Exceptions

In Java, an exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. It is an object that represents an error or an unexpected condition that a program encounters.

Types of Exceptions

There are two main types of exceptions in Java:

1. Checked Exceptions

Checked exceptions are exceptions that are checked at compile-time. They are typically used to handle errors that can be anticipated and recovered from, such as file not found or network connection errors.

2. Unchecked Exceptions

Unchecked exceptions are exceptions that are not checked at compile-time. They are typically used to handle programming errors, such as null pointer exceptions or array index out of bounds exceptions.

Subtypes of Exceptions

- i. Runtime Exceptions: These are unchecked exceptions that occur during runtime, such as `NullPointerException` or `ArrayIndexOutOfBoundsException`.
- ii. Error: These are unchecked exceptions that are thrown by the Java runtime environment, such as `OutOfMemoryError` or `StackOverflowError`.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            // Code that may throw an exception  
            int result = 10 / 0;  
        } catch (ArithmeticException e) {  
            // Handle the exception  
            System.out.println("Cannot divide by zero!");  
        }  
    }  
}
```

In this example, the `ArithmeticException` is a runtime exception that is thrown when attempting to divide by zero.

Benefits

- i. Error Handling: Exceptions provide a way to handle errors and unexpected conditions in a program.
- ii. Code Robustness: Exceptions help make code more robust by providing a way to anticipate and recover from errors.
- iii. Debugging: Exceptions provide valuable information about errors, making it easier to debug code.

By understanding exceptions and their types, you can write more robust and error-free Java code.

Exception Hierarchy

In Java, exceptions are organized in a hierarchical structure. The `Throwable` class is the root of the exception hierarchy, and all exceptions and errors inherit from it.

Hierarchy Structure

The exception hierarchy consists of the following levels:

1. `Throwable`: The root class of the exception hierarchy.
2. `Exception`: A subclass of `Throwable` that represents exceptions that can be thrown by a program.
3. `RuntimeException`: A subclass of `Exception` that represents runtime exceptions.
4. `Error`: A subclass of `Throwable` that represents errors that are thrown by the Java runtime environment.

Key Classes in the Hierarchy

- i. `Throwable`: The root class of the exception hierarchy.
- ii. `Exception`: The base class for all exceptions.
- iii. `RuntimeException`: The base class for all runtime exceptions.
- iv. `Error`: The base class for all errors.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            // Code that may throw an exception  
            throw new Exception("Something went wrong!");  
        } catch (Exception e) {  
            // Handle the exception  
        }  
    }  
}
```

```

        System.out.println("Caught exception: " + e.getMessage());
    }
}
}

```

In this example, the Exception class is used to represent a general exception.

Benefits

- i. Organized Exception Handling: The exception hierarchy provides a structured way to handle exceptions and errors.
- ii. Flexibility: The hierarchy allows for custom exceptions to be created and handled.
- iii. Robust Error Handling: The hierarchy enables robust error handling mechanisms to be implemented.

By understanding the exception hierarchy, you can design more effective exception handling mechanisms in your Java applications.

Throwing Exceptions

Throwing an exception is the process of explicitly generating an exception in a program. This can be done using the throw keyword.

Example

```

public class Main {
    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (Exception e) {
            System.out.println("Caught exception: " + e.getMessage());
        } }
    static void validateAge(int age) throws Exception {
        if (age < 18) {
            throw new Exception("Age must be 18 or above.");
        }
    }
}

```

In this example, the validateAge method throws an exception if the age is less than 18.

Catching Exceptions

Catching an exception is the process of handling an exception that has been thrown. This can be done using a try-catch block.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
}
```

In this example, the ArithmeticException is caught and handled in the catch block.

Best Practices

- i. Specific Exception Handling: Catch specific exceptions instead of general exceptions.
- ii. Meaningful Error Messages: Provide meaningful error messages when throwing exceptions.
- iii. Proper Resource Cleanup: Ensure that resources are properly cleaned up in the finally block.

Benefits

- i. Robust Error Handling: Throwing and catching exceptions enable robust error handling mechanisms.
- ii. Code Readability: Exceptions can improve code readability by separating error handling from regular code.
- iii. Flexibility: Exceptions provide flexibility in handling errors and exceptions.

By understanding how to throw and catch exceptions, you can design more robust and error-free Java applications.

Built-in Exceptions

Java has a range of built-in exceptions that can be used to handle common errors and exceptions. These exceptions are part of the Java API and can be used in a variety of situations.

Examples of Built-in Exceptions

- i. `ArithmeticException`: Thrown when an arithmetic operation is attempted that is not valid, such as dividing by zero.
- ii. `NullPointerException`: Thrown when an attempt is made to access or manipulate a null object reference.
- iii. `ArrayIndexOutOfBoundsException`: Thrown when an attempt is made to access an array element with an index that is outside the bounds of the array.
- iv. `FileNotFoundException`: Thrown when an attempt is made to access a file that does not exist.

Using Built-in Exceptions

Built-in exceptions can be used in a variety of situations, such as:

- i. **Error Handling**: Built-in exceptions can be used to handle errors and exceptions in a program.
- ii. **Validation**: Built-in exceptions can be used to validate input data and ensure that it is valid.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] array = new int[5];  
            System.out.println(array[10]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
}
```

In this example, an `ArrayIndexOutOfBoundsException` is thrown when an attempt is made to access an array element with an index that is outside the bounds of the array.

Benefits

- i. Robust Error Handling: Built-in exceptions provide a robust way to handle errors and exceptions in a program.
- ii. Code Readability: Built-in exceptions can improve code readability by providing a clear and concise way to handle errors.
- iii. Reusability: Built-in exceptions can be reused in a variety of situations, reducing the need to create custom exceptions.

By understanding built-in exceptions, you can write more robust and error-free Java code.

Creating Own Exceptions

In Java, you can create your own custom exceptions by extending the Exception class or one of its subclasses. Custom exceptions can be used to represent specific error conditions that are not covered by built-in exceptions.

Why Create Custom Exceptions?

- i. Specificity: Custom exceptions can provide more specific information about the error condition.
- ii. Readability: Custom exceptions can improve code readability by providing a clear and concise way to handle specific errors.
- iii. Reusability: Custom exceptions can be reused in multiple parts of a program.

Example

```
// Custom exception class
```

```
class InsufficientBalanceException extends Exception {  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}
```

```
public class BankAccount {  
    private double balance;
```

```
    public BankAccount(double balance) {  
        this.balance = balance;  
    }
```

```
    public void withdraw(double amount) throws InsufficientBalanceException {
```

```

    if (amount > balance) {
        throw new InsufficientBalanceException("Insufficient balance in account.");
    }
    balance -= amount;
}

public static void main(String[] args) {
    BankAccount account = new BankAccount(1000.0);
    try {
        account.withdraw(1500.0);
    } catch (InsufficientBalanceException e) {
        System.out.println("Caught exception: " + e.getMessage());
    }
}
}

```

In this example, a custom `InsufficientBalanceException` is created to represent a specific error condition.

Best Practices

- i. **Meaningful Names:** Choose meaningful names for custom exceptions.
- ii. **Clear Messages:** Provide clear and concise error messages.
- iii. **Proper Inheritance:** Extend the `Exception` class or one of its subclasses.

Benefits

- i. **Robust Error Handling:** Custom exceptions provide a robust way to handle specific error conditions.
- ii. **Improved Code Readability:** Custom exceptions can improve code readability by providing a clear and concise way to handle errors.
- iii. **Flexibility:** Custom exceptions provide flexibility in handling errors and exceptions.

By creating custom exceptions, you can design more robust and error-free Java applications that handle specific error conditions in a clear and concise way.

Stack Trace Element

A stack trace element in Java is an object that represents a single stack frame in the call stack. It provides information about the method call that led to an exception or error.

StackTraceElement Class

The StackTraceElement class in Java provides methods to access information about a stack frame, such as:

- i. `getClassName()`: Returns the name of the class where the exception occurred.
- ii. `getMethodName()`: Returns the name of the method where the exception occurred.
- iii. `getFileName()`: Returns the name of the file where the exception occurred.
- iv. `getLineNumber()`: Returns the line number where the exception occurred.

Example

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            method1();  
        } catch (Exception e) {  
            StackTraceElement[] stackTrace = e.getStackTrace();  
            for (StackTraceElement element : stackTrace) {  
                System.out.println("Class: " + element.getClassName());  
                System.out.println("Method: " + element.getMethodName());  
                System.out.println("File: " + element.getFileName());  
                System.out.println("Line: " + element.getLineNumber());  
            }  
        }  
    }  
  
    static void method1() {  
        method2();  
    }  
}
```

```

static void method2() {
    throw new RuntimeException("Test exception");
}
}

```

In this example, the `StackTraceElement` class is used to access information about the stack frames that led to an exception.

Benefits

- i. Debugging: Stack trace elements provide valuable information for debugging purposes.
- ii. Error Analysis: Stack trace elements help analyze the cause of errors and exceptions.
- iii. Logging: Stack trace elements can be used to log error information.

By understanding stack trace elements, you can effectively debug and analyze errors in your Java applications.

Input-Output Basics

Input-output (I/O) operations in Java involve reading and writing data to various sources, such as files, networks, and consoles. Java provides two types of streams for I/O operations: byte streams and character streams.

Byte Streams

Byte streams are used to read and write raw binary data, such as images, audio files, and other binary files. The `InputStream` and `OutputStream` classes are the base classes for byte streams.

Examples of Byte Streams

- i. `FileInputStream`: Used to read data from a file.
- ii. `FileOutputStream`: Used to write data to a file.

```
import java.io.FileInputStream;
```

```
import java.io.FileOutputStream;
```

```
import java.io.IOException;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        try (FileInputStream fis = new FileInputStream("input.txt");
```

```
            FileOutputStream fos = new FileOutputStream("output.txt")) {
```

```

byte[] buffer = new byte[1024];

int bytesRead;

while ((bytesRead = fis.read(buffer)) != -1) {
    fos.write(buffer, 0, bytesRead);
}

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}

}
}

```

Character Streams

Character streams are used to read and write text data, such as strings and characters. The Reader and Writer classes are the base classes for character streams.

Examples of Character Streams

- i. FileReader: Used to read text data from a file.
- ii. FileWriter: Used to write text data to a file.

```

import java.io.FileReader;

import java.io.FileWriter;

import java.io.IOException;

public class Main {

    public static void main(String[] args) {

        try (FileReader fr = new FileReader("input.txt");
            FileWriter fw = new FileWriter("output.txt")) {

            int character;

            while ((character = fr.read()) != -1) {

                fw.write(character);
            }
        }
    }
}

```

```

    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

Benefits

- i. Flexibility: Byte streams and character streams provide flexibility in handling different types of data.
- ii. Efficiency: Streams provide an efficient way to read and write data.
- iii. Readability: Streams improve code readability by providing a clear and concise way to handle I/O operations.

By understanding byte streams and character streams, you can effectively handle I/O operations in your Java applications.

Reading and Writing

Reading and writing are fundamental operations in Java that involve transferring data between different sources, such as files, networks, and consoles.

Reading Data

Reading data involves retrieving data from a source, such as a file or network connection. Java provides various classes and methods for reading data, including:

- i. `BufferedReader`: Used to read text data from a character stream.
- ii. `FileReader`: Used to read text data from a file.
- iii. `Scanner`: Used to parse primitive types and strings from a source.

Writing Data

Writing data involves sending data to a destination, such as a file or network connection. Java provides various classes and methods for writing data, including:

- i. `BufferedWriter`: Used to write text data to a character stream.
- ii. `FileWriter`: Used to write text data to a file.
- iii. `PrintWriter`: Used to write formatted data to a stream.

Example

```
import java.io.BufferedReader;

import java.io.BufferedWriter;

import java.io.FileReader;

import java.io.FileWriter;

import java.io.IOException;


public class Main {

    public static void main(String[] args) {

        try (BufferedReader br = new BufferedReader(new FileReader("input.txt"));

            BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {

            String line;

            while ((line = br.readLine()) != null) {

                bw.write(line);

                bw.newLine();

            }

        } catch (IOException e) {

            System.out.println("Error: " + e.getMessage());

        }

    }

}
```

In this example, data is read from a file using a `BufferedReader` and written to another file using a `BufferedWriter`.

Best Practices

- i. Use try-with-resources: Use try-with-resources statements to ensure that resources are properly closed.
- ii. Handle exceptions: Handle exceptions that may occur during reading and writing operations.
- iii. Use buffering: Use buffering to improve performance when reading and writing large amounts of data.

Benefits

- i. Efficient data transfer: Reading and writing operations enable efficient data transfer between different sources.
- ii. Flexibility: Java provides various classes and methods for reading and writing data, making it flexible and adaptable to different use cases.
- iii. Robustness: Proper handling of exceptions and resources ensures robustness and reliability in reading and writing operations.

By understanding reading and writing operations, you can effectively transfer data between different sources in your Java applications.

Console Reading and Writing Files

Console reading and writing files involve reading input from the console and writing output to files. Java provides various classes and methods for console reading and writing files.

Console Reading

Console reading involves reading input from the console using classes such as:

- i. Scanner: Used to parse primitive types and strings from the console.
- ii. BufferedReader: Used to read text data from the console.

Example of Console Reading

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");

        String name = scanner.nextLine();

        System.out.println("Hello, " + name + "!");

        scanner.close();

    }

}
```

Writing to Files

Writing to files involves sending data to a file using classes such as:

- i. `FileWriter`: Used to write text data to a file.
- ii. `BufferedWriter`: Used to write text data to a file with buffering.

Example of Writing to Files

```
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write("Hello, World!");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Reading from Files

Reading from files involves retrieving data from a file using classes such as:

- i. `FileReader`: Used to read text data from a file.
- ii. `BufferedReader`: Used to read text data from a file with buffering.

Example of Reading from Files

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("input.txt"))) {
```

```
String line;

while ((line = reader.readLine()) != null) {

    System.out.println(line);

}

} catch (IOException e) {

    System.out.println("Error: " + e.getMessage());

}

}

}
```

Benefits

- i. User interaction: Console reading enables user interaction and input.
- ii. Data persistence: Writing to files enables data persistence and storage.
- iii. Data retrieval: Reading from files enables data retrieval and processing.

By understanding console reading and writing files, you can create interactive and data-driven Java applications.